

LeadRadar — Sistem Rehberi

Yerel İşletme Lead Keşif & Zenginleştirme Sistemi • Uçtan uca çalışma mantığı • 08.10.2026

Bu belge, n8n üzerindeki "**Local Business Lead Discovery and Enrichment Agent**" (Marco'nun Lead Avcısı) iş akışının n8n **kullanılmadan**, bağımsız bir Python yazılımı olarak yeniden inşa edilmiş hâli olan **LeadRadar**'in uçtan uca çalışma mantığını anlatır. Amaç: haftada bir çalışıp Berlin'deki küçük işletmelerden (kuaför, restoran, fitness salonu, diş hekimi) web sitesi olmayan ya da sorunlu olanları bulmak, kanıta dayalı biçimde puanlamak ve satış görüşmesine hazır bir PDF raporu üretmektir.

1. Sistem Ne Yapar? (Kullanım Senaryosu)

Hedef kullanıcı, bir şehirde çalışan serbest web tasarımcısı / ajans sahibidir. Müşteri adayı bulmak için her hafta şu soruyu sorar: "*Hangi yerel işletmelerin web sitesi yok ya da sitesi müşteri kaybettiriyor?*" LeadRadar bu soruyu otomatik yanıtlar:

- Her hafta farklı kategori ve semt kombinasyonunu tarar (rotasyon).
- İşletmeleri; adı, adresi, telefonu ve web sitesi kaydıyla birlikte keşfeder.
- Önceki haftalarda bulunanları eler — her hafta **yeni** adaylar gelir.
- Sitesi olan adayların sitesini teknik olarak denetler (HTTPS, mobil uyum, iletişim yolları, bozuk linkler).
- Her adayı YÜKSEK / ORTA / DÜŞÜK fırsat skoruyla puanlar — yalnızca kanıtlar.
- Sonuçları veritabanına kaydeder ve satışa hazır bir PDF raporu üretir.

2. Uçtan Uca Akış Şeması



3. Adım Adım Çalışma Mantığı

Adım 1 — Haftalık Tetik ve Rotasyon (rotation.py)

Orijinal sistem her pazartesi 09:00'da tetiklenir. LeadRadar'de aynı işi **Windows Görev Zamanlayıcı** yapar (bkz. Bölüm 8) ya da **py run.py** komutuyla elle çalıştırılır. Yılın kaçınıcı haftasında olduğumuz hesaplanır ve **hafta % 4** formülüyle 4 hazır rotasyondan biri seçilir. Böylece her hafta farklı 3 kategori + 3 semt taranır ve aynı işletmeler sürekli karşınıza çıkmaz. Rotasyon listeleri orijinal iş akışındaki 'Rotation Seed' düğümünün birebir aynısıdır.

Hafta 29 → kategoriler: Fitnessstudio, Zahnarzt, Friseur | semtler: Charlottenburg, Wilmerdorf, Schöneberg

Adım 2 — Keşif (discover.py)

Orijinal akışta bir yapay zeka ajanı, Firecrawl'un /search ve /scrape araçlarıyla yelp.de ve gelbeseiten.de gibi rehberleri tarıyordu. LeadRadar bu adımı **OpenStreetMap Overpass API** ile yapar: ücretsizdir, API anahtarı gerektirmez ve işletme adı, adresi, telefonu ve **web sitesi kaydı** gibi yapısal veriyi doğrudan verir. Her semt için tek sorgu atılır; sonuçlar kategoriye eşlenir (ör. shop=hairstylist → Friseur). Kanıt linki olarak işletmenin OSM kaydı saklanır. OSM kaydında marka (brand) etiketi olan işletmeler ve bilinen zincir isimleri otomatik elenir — orijinaldeki 'zincir/franchise dışı' kuralı.

Adım 3 — Parmak İzi ve Tekilleştirme (fingerprint.py + store.py)

Her aday için normalize edilmiş bir tekilleştirme anahtarı üretilir; öncelik sırası orijinaldekiyle aynıdır:

isim|adres → isim|telefon → isim|site-alan-adı → isim|

SQLite'taki **marco_leads** tablosunda saklanan tüm eski parmak izleri çekilir ve havuzdan daha önce görülen adaylar elenir. Bu, orijinal akıştaki 'Compute Fingerprint' + 'Fetch Seen Fingerprints' + 'Filter Unseen (Top 10)' üçlüsünün karşılığıdır.

Adım 4 — Seçim Önceliği

Orijinal ajan talimatı: "20 adayın en az 12'si web sitesi olmayan işletme olsun; kalanlar zayıf web varlığı olanlardan seçilsin." LeadRadar aynı oranı korur: seçilen 10 adayın ~%60'ı web sitesi hiç olmayanlardan, kalanı önce 'yalnızca sosyal medya sayfası olanlar', sonra 'http://' (güvensiz) site kullananlar, en son normal siteli işletmelerden doldurulur. Kategori ve semt çeşitliliği korunur (round-robin).

Adım 5 — Web Sitesi Denetimi (audit.py)

Orijinal akışta ikinci bir yapay zeka ajanı siteyi Firecrawl ile kazıyıp değerlendiriyordu. LeadRadar bu değerlendirmeyi **deterministik kodla** yapar — böylece sonuçlar tekrarlanabilir ve halüsinasyonsuzdur. Denetlenen **nesnel engeller** (objective blockers) orijinal guardrail listesinin birebir aynısıdır:

- **Site yüklenmiyor** — DNS/bağlantı hatası, HTTP 4xx/5xx, boş sayfa, sonsuz yönlendirme
- **HTTPS yok** — site şifrelenmemiş http üzerinden sunuluyor ya da SSL sertifikası geçersiz
- **Mobil uyumsuz** — sayfada viewport meta etiketi yok
- **İletişim yolu yok/bozuk** — telefon, e-posta, iletişim sayfası bulunamıyor ya da linki 404 veriyor
- **Randevu/rezervasyon yolu bozuk** — sitedeki randevu linki hata veriyor

Bunlara ek olarak **sezgisel** (heuristic) bulgular toplanır ve '[HEURISTIC]' önekiyle ayrı tutulur: eski telif yılı, çok eski jQuery/CMS sürümü, eksik SEO etiketleri vb. Ayrıca sitede bulunan telefonlar, e-postalar, sosyal medya linkleri ve teknoloji sinyalleri (WordPress, Wix, Jimdo...) çıkarılır — bunlar rapordaki iletişim planını besler.

Adım 6 — Puanlama: Guardrail Kuralları (score.py)

Skorlama, orijinal sistem mesajındaki '**OPPORTUNITY SCORING — STRICT GUARDRAILS**' kurallarının koda dökülmüş hâlidir:

Durum	Skor
Web sitesi hiç yok (kaynakta site listelenmemiş)	YÜKSEK — birebir hedef profil
Resmî site yerine yalnızca Facebook/Instagram sayfası	YÜKSEK — 'resmî site yok' sayılır (belgelenmiş ek karar)
Site var + en az 2 doğrulanmış nesnel engel	YÜKSEK
Site var + tam 1 nesnel engel	ORTA
Site var + 0 engel ama 2+ sezgisel bulgu	ORTA
HTTPS var + çalışan iletişim + bariz hata yok	ORTA veya DÜŞÜK (asla YÜKSEK olamaz)
Yalnızca sezgisel bulgular ('site eski görünüyor')	Asla YÜKSEK üretemez — orijinal 3. kural

why_lead (neden aday?) alanı daima kanıta dayalıdır: ya rehber gerçeği ('site kaydı yok') ya da doğrulanmış 1-2 site sorunu yazılır. Sezgisel izlenimler bu alana değil, ayrı 'sezgisel değerlendirme' bölümüne gider — orijinal 5. kural.

Adım 7 — Kayıt (store.py)

Zenginleştirilen her aday SQLite'taki **marco_leads** tablosuna yazılır. Tablo şeması, orijinal iş akışındaki yapışkan notta verilen Postgres SQL'inin birebir SQLite karşılığıdır (+ 'district' ve 'emails' ek sütunları). Aynı parmak izi ikinci kez gelirse sessizce atlanır (INSERT OR IGNORE) — orijinaldeki continueOnFail davranışı. Bu tablo sistemin hafızasıdır: gelecek haftaların tekilleştirmesi buradan beslenir.

Adım 8 — PDF Raporu (report.py)

Orijinal akış sonuçları HTML e-posta olarak Gmail ile gönderiyordu. LeadRadar bunun yerine **output/** klasörüne bir PDF raporu üretir. Rapor önce çalıştırma parametrelerini ve skora göre sıralı özet tabloyu (YÜKSEK → DÜŞÜK, orijinal e-postadaki sıralama) gösterir; ardından her aday için bir sayfa ayrılır ve şu sırayla sunulur:

- **1) Parametreler** — kategori, semt, skor, faaliyet durumu, güncellik ve teknoloji sinyalleri
- **2) Bilgiler ve İletişim** — adres, telefon, sitede bulunan e-postalar, sosyal medya, kanıt linki ve öncelik sıralı 'nasıl iletişime geçilir' planı + görüşme açılış önerisi
- **3) Web Sitesi Hata ve Sorunları** — doğrulanmış nesnel sorunlar, sezgisel bulgular ve teknik denetim ayrıntı tablosu

Ayrıca aynı klasöre, tüm ham verinin döküldüğü bir JSON dosyası yazılır (şeffaflık ve hata ayıklama için).

4. Orijinal n8n Düğümleri ↔ LeadRadar Modülleri

Orijinal n8n düğümü	LeadRadar karşılığı
Schedule Trigger (Pzt 09:00 cron)	Windows Görev Zamanlayıcı / elle py run.py
Rotation Seed (Code)	leadradar/rotation.py — aynı hafta formülü, aynı listeler
AI Agent — Discover Fresh (GPT-5.1)	leadradar/discover.py — Overpass API ile deterministik keşif
/search + /scrape in Firecrawl (araçlar)	Overpass (temel keşif) + Firecrawl (açıksa: rehberlerden puan/yorum + JS'li site tarama) — leadradar/firecrawl_client.py
Parse Refill Output (Code)	discover.py içindeki alan eşleme (element_to_lead)
Compute Fingerprint (Code)	leadradar/fingerprint.py — aynı normalizasyon ve öncelik
Fetch Seen Fingerprints (Postgres)	leadradar/store.py — SQLite SELECT fingerprint
Filter Unseen Top 10 (Code)	run.py — havuzdan yeni olanları seçme + select_final
Enrich Prep / Parse (Code)	run.py döngüsü + score.py alan üretimi
AI Agent — enrichment (GPT-5.1)	leadradar/audit.py + score.py — kural tabanlı denetim ve guardrail puanlama
Postgres — Insert Lead	store.py insert_lead (INSERT OR IGNORE)
Aggregate + Build Email HTML	leadradar/report.py — PDF üretimi
Gmail — Send Weekly Summary	output/LeadRadar_Lead_Raporu_<tarih>.pdf dosyası

Neden yapay zeka ajanları yerine kural tabanlı kod? Orijinaldeki guardrail talimatlarının neredeyse tamamı zaten nesnel, kodla doğrulanabilir kontrollerdi (HTTPS var mı, viewport var mı, iletişim linki çalışıyor mu...). Kod bu kontrolleri halüsinasyon riski olmadan, API maliyeti olmadan ve her seferinde aynı sonuçla yapar. İsteğe bağlı yapay zeka cılası (Bölüm 7) yine de mevcuttur.

Firecrawl entegrasyonu (isteğe bağlı, config.json → firecrawl.enabled). Orijinal akıştaki Firecrawl araçları bu sistemde de kullanılabilir. Açıkken iki ek yetenek devreye girer: (1) her aday için yelp.de / gelbeseiten.de / jameda.de gibi rehberler taranarak **puan ve yorum sayısı** doldurulur (OSM bu veriyi tutmaz); (2) yerel tarayıcı ince/boş içerik gördüğünde (JavaScript ile yüklenen siteler) site **Firecrawl ile yeniden taranır** ve gerçek içerik denetlenir. Anahtar önce FIRECRAWL_API_KEY ortam değişkeninde, yoksa Firecrawl CLI'nin kaydettiği dosyada aranır — yani firecrawl CLI ile giriş yaptıysanız ek ayar gerekmez. Firecrawl kapalıyken sistem tamamen ücretsiz ve eskisi gibi çalışır; skorlar her iki durumda da kural tabanlıdır.

5. Veritabanı Şeması (data/leads.db)

```
CREATE TABLE marco_leads (  
  id INTEGER PRIMARY KEY, fingerprint TEXT UNIQUE,  
  business_name TEXT NOT NULL, category TEXT, address TEXT,  
  phone TEXT, website TEXT, public_rating TEXT, review_count TEXT,  
  website_issues TEXT, heuristic_assessment TEXT, services_listed TEXT,  
  tech_signals TEXT, freshness_signal TEXT, operating_status TEXT,  
  opportunity_score TEXT, why_lead TEXT, source_links TEXT,  
  week_of TEXT, discovered_at TEXT, district TEXT, emails TEXT )
```

Veriyi incelemek için herhangi bir SQLite aracı (ör. DB Browser for SQLite) kullanılabilir.

6. Kurulum ve Çalıştırma

- Gereksinim: Python 3.10+ (bu makinede 3.14 kurulu) ve internet bağlantısı.

```
py -m pip install -r requirements.txt
cd C:\Users\furka\Music\LeadRadar
py run.py # tam haftalık çalıştırma
py run.py --limit 5 # daha az adayla hızlı deneme
py run.py --week 31 # başka bir rotasyon haftasını zorla
```

Çıktılar: **output/LeadRadar_Lead_Raporu_<tarikh>.pdf** (rapor), **output/LeadRadar_calistirma_<tarikh>.json** (ham veri), **data/leads.db** (hafıza).

7. Yapılandırma (config.json)

Ayar	Açıklama
city	Hedef şehir (varsayılan: Berlin). Overpass sayesinde başka şehirler de çalışır; semtler için area_rotation ve idari seviyeler için *_admin_levels ayarlanmalıdır.
target_leads / discover_pool	Rapora seçilecek aday sayısı (10) / keşif havuzu (20) — orijinal oranlar.
no_website_ratio	Seçimde 'web sitesi yok' kotası (0.6 = %60).
category_rotation / area_rotation	Haftalık rotasyon listeleri — orijinalle birebir aynı.
category_osm	Kategori → OSM etiket eşlemesi. Yeni kategori eklemek için buraya satır eklenir.
chain_blacklist	Zincir/franchise isim filtresi (brand etiketi olanlar zaten otomatik elenir).
request_delay_sec / http_timeout	Nazik tarama: istekler arası bekleme ve zaman aşımı.
firecrawl.enabled	İsteğe bağlı Firecrawl zenginleştirilmesi (puan/yorum + JS'li site tarama).
firecrawl.enrich_ratings / scrape_sites	Sırasıyla: rehberden puan çekme / ince siteleri Firecrawl ile tarama.
ai.enabled / ai.provider	İsteğe bağlı yapay zeka cilasası (Bölüm 7.1).

7.1 İsteğe bağlı yapay zeka cilasası (ai_polish.py)

Orijinal akıştaki gibi metinlerin bir dil modeliyle zenginleştirilmesini isterseniz: config.json içinde **"ai"**: **{"enabled": true}** yapın ve ortam değişkeni olarak **ANTHROPIC_API_KEY** (veya provider=openai için **OPENAI_API_KEY**) tanımlayın. Bu durumda her adayın 'neden aday?' gerekçesi ve sezgisel değerlendirmesi, denetim kanıtlarına dayanarak dil modeli tarafından yeniden yazılır. Kapalıyken sistem tamamen ücretsiz ve çevrimdışı puanlama mantığıyla çalışır; skorlar her iki durumda da kural tabanlıdır (model skoru değiştiremez).

8. Haftalık Otomatik Çalıştırma (Pazartesi 09:00)

Orijinal akıştaki cron tetikleyicisinin (0 9 * * 1) Windows karşılığını kurmak için proje klasöründeki hazır betiği **bir kez** çalıştırın:

```
powershell -ExecutionPolicy Bypass -File haftalik_zamanlama.ps1
```

Betik, Görev Zamanlayıcı'da her pazartesi 09:00'da **py run.py** komutunu çalıştıran 'LeadRadar Lead Avcısı' adlı görevi oluşturur. Kaldırmak için: **schtasks /Delete /TN "LeadRadar Lead Avcısı" /F**

9. Orijinalden Farklar ve Sınırlar (Dürüst Liste)

Orijinal	LeadRadar yaklaşımı
Firecrawl + GPT-5.1 keşfi	Overpass API (ücretsiz, anahtarsız). Yelp/GelbeSeiten puanları alınamadığı için public_rating / review_count alanları boş kalır.
'Rehberde site yok' kanıtı	OSM kaydında site alanının boş olması kanıt sayılır. OSM eksik olabilir; işletmenin aslında sitesi olabilir — aramadan önce hızlı bir Google kontrolü önerilir.
Yapay zeka değerlendirmesi	Kural tabanlı denetim + isteğe bağlı yapay zeka cılası. Skorlar deterministiktir, tekrarlanabilir.
Postgres (Supabase)	SQLite — kurulum gerektirmez, tek dosya (data/leads.db).
Gmail e-postası	PDF raporu (output/ klasörü).
Faaliyet durumu (yorum tazeliği)	Yorum verisi olmadığından telefon/çalışma saati/site erişimi gibi dolaylı sinyallerle tahmin edilir ve raporda 'teyit önerilir' diye işaretlenir.

Etik not: Sistem yalnızca halka açık işletme bilgilerini kullanır, sitelere saniyeler arayla tek tek istek atar (nazik tarama) ve hiçbir yere otomatik mesaj göndermez — iletişim kararı her zaman size aittir.

10. Sık Sorulanlar / Sorun Giderme

- **Rapor boş geldi:** O haftanın semtlerinde yeni aday kalmamış olabilir. --week ile başka rotasyon deneyin ya da data/leads.db dosyasını silerek hafızayı sıfırlayın.
- **Overpass hatası (429/504):** Sunucu yoğun; sistem otomatik bekleyip yedek uca geçer. Sorun sürerse birkaç dakika sonra tekrar çalıştırın.
- **Türkçe karakterler bozuk görünüyor:** PDF'te görünmez (gömülü Segoe UI/Arial kullanılır); yalnızca konsol çıktısında görülebilir, zararsızdır.
- **Başka şehir istiyorum:** config.json'da city + area_rotation değerlerini değiştirin (ör. city=İstanbul, semtler=Kadıköy, Beşiktaş...; city_admin_levels için Türkiye'de '4', district_admin_levels için '6|8' iyi bir başlangıçtır).
- **Aday sayısını artırmak:** config.json'da target_leads ve discover_pool değerlerini yükseltin.